

Bmake It

A BSD-Make-Based Cross-Platform Build System
Package Documentation

Draft v1

September 7, 2026

Abstract

This document describes **Bmake It**, a BSD-make-based build system for C, C++, and lex/yacc grammar sources, designed to mimic the conceptual model of Dassault Systèmes' RADE/**mkmk** tool (workspaces, frameworks, modules, prerequisites) while reusing real BSD make infrastructure wherever it fits¹. Every substantive claim in this document is footnoted with the requirement, decision, or constraint alias from the project's discovery-driven design record (**project-model/**) that established it, so any statement here can be traced back to its origin and rationale.

This is a *draft* document, written while the design is still being finalized; sections marked **[open]** describe areas where the underlying design is intentionally still undecided or deferred.

¹CON-use-bsdmake

Contents

1	Overview	3
1.1	What Bmake It Is	3
1.2	Design Philosophy	3
1.3	The Domain Model	4
1.4	Platform and Language Scope	5
2	System Requirements	6
2.1	Build System	6
2.2	Toolchain	6
2.3	Languages	7
2.4	Scope	7
3	Basic Usage	8
3.1	Directory Layout at a Glance	8
3.2	Creating a Workspace	8
3.3	Creating a Framework	9
3.4	Creating a Module	9
3.5	Header Visibility	10
3.6	Building	10
3.7	Cleaning and Installing	11
4	Advanced Features	12
4.1	Target Selection and Cross-Compilation	12
4.2	Resource Files	13
4.3	Generated Headers	13
4.4	Versioned Shared Libraries	13
4.5	Packaging	14
4.6	Running a Built Program	14
4.7	Self-Documentation: <code>make help</code>	14
4.8	Convenience Targets	15
4.9	Dependency-Order Generation	15
4.10	Caching	16
4.11	Empirical Validation Summary	16
5	Tutorial: Building System and Hello From Scratch	18
5.1	Step 1 — Create the Workspace	18
5.2	Step 2 — Create the <code>System</code> Framework	18
5.3	Step 3 — Create the <code>Hello</code> Framework	18
5.4	Step 4 — Create the <code>libgreet.m</code> Library Module	19

5.5	Step 5 — Create the <code>hello.m</code> Executable Module	19
5.6	Step 6 — Build	19
5.7	Step 7 — Run It	20
5.8	Step 8 — Clean Up	20

Chapter 1

Overview

1.1 What Bmake It Is

Bmake It is a cross-platform build system for C and C++ (plus lex/yacc grammar sources, which compile down to C/C++)¹, built on top of real BSD make rather than GNU make or a bespoke build tool. Its object model — *workspaces* containing *frameworks* containing *modules* — is deliberately modeled on Dassault Systèmes’ RADE/**mkmk** tool, a CAA (Component Application Architecture) build system used internally at Dassault Systèmes since the late 1990s. Where **mkmk**’s specific mechanisms don’t fit BSD make’s own idioms well, **Bmake It** diverges deliberately rather than reproducing them exactly².

Three mechanisms present in real **mkmk** are explicitly *not* carried over, by design rather than oversight:

- No dedicated “identity card” special file for declaring prerequisites — prerequisites are declared directly in the ordinary framework **makefile**³.
- No four-level RADE header hierarchy (Public / Protected / Private / Local) — **Bmake It** uses a simpler three-level scheme instead, described in Chapter 3⁴.
- No **mkGetPreq**/**mkCopyPreq**-style commands for managing prerequisite search paths — prerequisite resolution is purely dynamic, with no static-copy mode and no dedicated setup command⁵.

1.2 Design Philosophy

The guiding principle throughout is: *reuse real BSD make variables and mechanisms wherever they already exist; introduce new, project-specific ones only where BSD make genuinely has no equivalent.* This shows up consistently:

¹REQ-supported-languages

²CON-use-bsdmake

³NREQ-no-identitycard-special-file

⁴NREQ-no-four-level-header-visibility

⁵NREQ-no-copy-preq-commands

- Target selection reuses BSD make’s own `TARGET=/TARGET_ARCH=` convention rather than inventing new names⁶.
- Installation reuses `DESTDIR` unchanged; `PREFIX`, it turns out, is not actually a native BSD make concept at all (see Section 4.11)⁷.
- Compile and link flags (`CFLAGS`, `CXXFLAGS`, `LDFLAGS`, `YFLAGS`, `LFLAGS`) are the ordinary BSD make variables, extended via `+=` — no wrapper macros the way real `mkmk` used `LOCAL_CFLAGS`-style names⁸.
- Debug-vs-release is controlled via BSD make’s own `DEBUG_FLAGS`, not a new `MODE=debug|release` abstraction⁹.
- Shared-library versioning reuses `SHLIB_MAJOR`/`SHLIB_MINOR` directly from real `bsd.lib.mk`¹⁰.

New, project-specific concepts (`PARENT_WS`, `PREREQS`, `PROG`, `LIB`, `LIBS`, `LIB_SHARED`, `INCL`, `TOOLCHAIN`) exist only for domain concepts BSD make has no native equivalent for at all — workspace parenting, framework prerequisites, module identity, linking, header promotion, and toolchain bundle selection¹¹.

This reuse-first philosophy has real, tested limits. An empirical validation pass against real FreeBSD `share/mk` and a portable `bmake` binary found that several assumed “native reuse” cases were actually wrong: `LIB_SHARED` has no native BSD-make equivalent at all (real `bsd.lib.mk` builds a static archive whenever `LIB` is set, and *additionally* builds a shared library only if `SHLIB_MAJOR` is present)¹², `PREFIX` does not exist in `bsd.prog.mk`/`bsd.lib.mk` at all¹³, and `TARGET`/`TARGET_ARCH` have no effect whatsoever without FreeBSD’s full base-system build machinery, which a standalone project does not have¹⁴. These corrections are described in Section 4.11 and are already reflected in the macro reference in Chapter 4.

1.3 The Domain Model

Workspace A set of frameworks¹⁵. A workspace may declare an ordered list of *parent* workspaces; the build searches them, in declared order, for anything not found locally¹⁶. This search is purely dynamic — there is no physical-copy mode and no separate setup command, unlike real `mkmk`’s `mkGetPreq`/`mkCopyPreq`¹⁷. A workspace carries no resource files of its own¹⁸.

Framework Contains one or more modules, exposes public headers, and declares its prerequisite frameworks¹⁹. A framework’s prerequisite declaration is *mandatory*, even when empty — its

⁶REQ-target-arch-toolchain-selection-is-custom-req

⁷REQ-prefix-is-new-not-reused-req

⁸REQ-flags-reuse-bsd-native-vars-req

⁹REQ-debug-release-via-native-bsd-flags-req

¹⁰REQ-shlib-major-minor-cross-platform-emission-req

¹¹REQ-uniform-makefile-filename-req

¹²REQ-lib-shared-is-new-not-reused-req

¹³REQ-prefix-is-new-not-reused-req

¹⁴REQ-target-arch-toolchain-selection-is-custom-req

¹⁵REQ-workspace-is-set-of-frameworks

¹⁶REQ-workspace-parent-search-order

¹⁷DEC-prereq-access-dynamic-only

¹⁸NREQ-no-workspace-level-share-source

¹⁹REQ-framework-contains-modules

presence is what makes a directory auto-discoverable as a framework in the first place²⁰.

Module The basic link unit: a module produces exactly one executable, static library, or shared library²¹. A module’s directory is named `NAME.m`²², and modules are auto-discovered by scanning — no explicit list is maintained by hand²³.

Resource files Non-source runtime data (icons, configuration files, and similar), carried by both modules and frameworks²⁴, staged through a three-layer `share/` overlay described in Section 4.2.

1.4 Platform and Language Scope

Languages: C, C++, and lex/yacc grammar sources. Java, Fortran, IDL, and Express — all supported by real `mkmk` — are explicitly out of scope for now²⁵.

Hosts: macOS (via MacPorts), Linux, and the *BSD family²⁶, plus Cygwin and WSL treated as *NIX-like build hosts (WSL is, for build purposes, simply Linux; no native MSVC toolchain is required)²⁷.

Phase 1 — the concrete, currently-targeted scope — is exactly four build targets²⁸:

Target key	Meaning
<code>macos-arm64</code>	macOS, Apple Silicon, LLVM (default toolchain)
<code>macos-arm64-gcc</code>	macOS, Apple Silicon, GCC
<code>freebsd-amd64</code>	FreeBSD, x86-64, LLVM (default toolchain)
<code>freebsd-amd64-gcc</code>	FreeBSD, x86-64, GCC

Everything outside this set — OpenBSD/NetBSD, per-distribution Linux labeling, kernel-level or Darwin/XNU development, macOS `.app` bundle output, the exact `win` ABI story (MinGW vs. Cygwin vs. MSVC), `musl` `libc` — is deliberately deferred, not designed against yet²⁹.

²⁰REQ-prereqs-mandatory-even-empty-req

²¹REQ-module-is-basic-link-unit

²²REQ-module-dir-naming-convention

²³REQ-modules-and-frameworks-auto-discovered-req

²⁴REQ-resource-file-model

²⁵REQ-supported-languages

²⁶CON-cross-platform-targets

²⁷REQ-windows-hosts-cygwin-wsl

²⁸REQ-phase1-concrete-targets

²⁹REQ-userland-only-scope

Chapter 2

System Requirements

This chapter states what must already be present on a build host before `Bmake` It can be used. For which hosts and languages are in scope at all, see Section 1.4.

2.1 Build System

A real BSD make binary is required. `Bmake` It is built on top of BSD make's own `bsd.*.mk` infrastructure, or reprogrammed equivalents where needed; it does not run under GNU make¹.

[open] Which package supplies BSD make on each host, and the exact command to install it, is not yet part of the design record. The `*BSD` family ships it as the system default, and macOS reaches it via MacPorts²; the situation on Linux is not yet documented here.

2.2 Toolchain

Phase 1 supports two toolchain bundles, selected via `TOOLCHAIN=<gcc|llvm>` on the `make` command line; if unspecified, it defaults to `llvm`³. At least one of LLVM (Clang) or GCC must be installed and reachable — see the Phase 1 target table in Section 1.4 for which combinations are currently targeted.

Caveat: toolchain detection currently locates the compiler via a single hardcoded absolute path (`/usr/bin/clang`, `/usr/bin/gcc`) rather than a `PATH` search. On a host where the compiler lives elsewhere — MacPorts' `/opt/local/bin`, pkgsrc's `/usr/pkg/bin`, Homebrew, or similar — `TOOLCHAIN=llvm` silently falls back to whatever plain `cc` happens to resolve to, with no warning that the requested toolchain was not actually used⁴.

¹CON-use-bsdmake

²CON-cross-platform-targets

³REQ-toolchain-cli-variable-req-v4

⁴OBS-toolchain-detection-hardcoded-path-silently-wrong-compiler

2.3 Languages

Bmake It compiles C, C++, and lex/yacc grammar sources (which themselves compile down to C/C++); no other language toolchain is required⁵.

2.4 Scope

Bmake It targets userspace binaries and libraries only — no kernel module or driver toolchain is needed⁶.

⁵REQ-supported-languages

⁶REQ-userland-only-scope

Chapter 3

Basic Usage

This chapter covers the minimum needed to create a workspace, populate it with a framework and a module, and build it.

3.1 Directory Layout at a Glance

Every level of the hierarchy — workspace, framework, module — uses a file literally named `makefile` (not `makefile.mk`), which `.includes` a role-specific helper file, mirroring the way real BSD `make`'s own `bsd.prog.mk/bsd.lib.mk` work¹:

Level	Makefile contents (first line)	Key macro(s)
Workspace	<code>.include <mk.workspace.mk></code>	<code>PARENT_WS=</code>
Framework	<code>.include <mk.framework.mk></code>	<code>PREREQS=</code>
Module (executable)	<code>.include <mk.prog.mk></code>	<code>PROG=, LIBS=</code>
Module (library)	<code>.include <mk.lib.mk></code>	<code>LIB=, LIB_SHARED=, LIBS=</code>

3.2 Creating a Workspace

A workspace is just a directory containing a `makefile`:

```
.include <mk.workspace.mk>
PARENT_WS=/abs/path/to/parent-ws-1 /abs/path/to/parent-ws-2
```

`PARENT_WS` is an ordered, space-delimited list of **absolute** paths to parent workspaces² — relative paths were explicitly considered and rejected as too fragile. If left empty, the workspace has no parents and everything it needs must live inside it. Frameworks inside the workspace are auto-discovered: any immediate subdirectory containing a `makefile` that declares `PREREQS=` is treated as a framework³; there is no explicit list to maintain.

¹REQ-uniform-makefile-filename-req

²REQ-workspace-parent-macro-absolute-req

³REQ-framework-discovery-signal-req

3.3 Creating a Framework

A framework is a directory (any name — no special suffix, unlike modules) containing its own makefile:

```
.include <mk.framework.mk>
PREREQS=System
```

PREREQS= is **mandatory even when empty** (PREREQS= with nothing after it is valid and means “no prerequisites”)⁴ — this is not merely documentation, it is the literal signal the workspace uses to recognize the directory as a framework at all. The list is ordered, all entries are implicitly public (there is no per-entry visibility qualifier the way real `mkmk`’s identity card distinguished Public/Protected/Private), and `Bmake It` does *not* validate the ordering or detect cycles⁵: an incorrectly ordered or cyclic PREREQS list simply surfaces later as an ordinary “header not found” compile error⁶.

A framework has three directories for hand-written headers, described fully in Section 3.5, and an optional `share/` directory for resources (Section 4.2).

3.4 Creating a Module

A module’s directory is always named `NAME.m`; `NAME` also becomes the default output name if not overridden⁷. Whether `NAME.m` and `name.m` are distinct depends entirely on the host filesystem’s own case sensitivity — `Bmake It` does not itself enforce or normalize case⁸.

An executable module

```
.include <mk.prog.mk>
PROG=hello # optional -- defaults to the directory name
LIBS=greet # link-time only, space-delimited
```

A library module

```
.include <mk.lib.mk>
LIB=greet # optional -- defaults to the directory name
LIB_SHARED=YES # default; LIB_SHARED=NO builds a static archive instead
LIBS=
```

`LIB_SHARED` defaults to `YES`; setting it explicitly to `NO` builds a static/archive library instead⁹. **Important, and confirmed by empirical testing against real `bsd.lib.mk`:** this `YES/NO` toggle is not itself a native BSD make mechanism — real `bsd.lib.mk` unconditionally builds a static archive whenever `LIB` is set, and additionally builds a shared library only when `SHLIB_MAJOR`

⁴REQ-prereqs-mandatory-even-empty-req

⁵REQ-framework-prereqs-macro-req

⁶REQ-prereqs-misorder-no-validation-req

⁷REQ-module-dir-naming-convention,REQ-prog-lib-default-to-module-name-req

⁸REQ-module-name-case-sensitivity-req

⁹REQ-lib-shared-is-new-not-reused-req

is also present. `Bmake` It's `mk.lib.mk` implements the `LIB_SHARED` abstraction on top of that native behavior¹⁰.

A module's `src/` directory is scanned automatically: every source file physically present there is treated as compilation input, with no `SRCS=`-style list to maintain by hand¹¹. `.y` and `.l` (yacc and lex) files are compiled through the same automatic mechanism, confirmed working via built-in suffix transformation rules with zero explicit rule declarations¹².

`LIBS=` lists other modules or system libraries this module links against. It is **purely a link-time instruction** and has no effect whatsoever on header visibility, even for sibling modules in the same framework¹³ — see Section 3.5 for how header visibility actually works.

3.5 Header Visibility

Headers have exactly three visibility levels¹⁴:

Level	Directory	Visible to
Public	<code>fw/include/</code>	Any framework declaring this one in <code>PREREQS=</code>
Framework-internal	<code>fw/local/include/</code>	All of this framework's own modules
Module-private	<code>module.m/include/</code>	Only that module's own source files

Header visibility is resolved **exclusively** through the containing framework's `PREREQS=` declaration — never through a module's `LIBS=`¹⁵. This is a genuine, deliberate design point worth internalizing: a module can link against a library via `LIBS=` while remaining completely unable to see that library's headers, if the containing frameworks aren't in a `PREREQS=` relationship. Headers are always read directly from source (`include/` directories) — never from build output — because headers are plain text and don't need per-target staging the way compiled artifacts do¹⁶. The one exception is *generated* headers (from yacc/lex), covered in Section 4.3.

3.6 Building

Run `make` in a module directory to build just that module, in a framework directory to build all its modules, or at the workspace root to build everything. Modules and frameworks are auto-discovered by scanning — modules via the `.m` suffix, frameworks via the `makefile+PREREQS=` signal¹⁷.

The build strategy is *fused*: each module compiles, generates headers if applicable, and links in one pass — matching real `bsd.*.mk` idiom directly, rather than a global compile-everything-then-link-everything split¹⁸. Build order across frameworks follows the `PREREQS=` dependency graph; build order across sibling modules follows the `LIBS=` graph plus any consumed generated-header

¹⁰REQ-lib-shared-is-new-not-reused-req

¹¹REQ-module-src-auto-discovery

¹²OBS-validation-yacc-lex-suffix-rules-confirmed

¹³REQ-libs-macro-link-only-headers-via-prereqs-req

¹⁴REQ-three-level-header-visibility-req

¹⁵REQ-headers-resolved-from-source-req

¹⁶REQ-headers-resolved-from-source-req

¹⁷REQ-modules-and-frameworks-auto-discovered-req

¹⁸REQ-fused-build-per-module-full-dependency-order-req

dependency¹⁹. The concrete mechanism generating this order — confirmed by a working prototype, not merely designed on paper — is described in Section 4.9.

```
$ cd myworkspace/Hello
$ make # builds every module in Hello, in dependency order
$ cd hello.m
$ make # builds just this one module
```

3.7 Cleaning and Installing

`make clean` removes **every** generated artifact for the selected target — not just the compiled binaries, but also the `distrib/` packaging staging area, per-module `.depend` files, and the internal dependency-resolution caches²⁰. After `make clean TARGET=all`, the tree should be indistinguishable from a fresh checkout.

`make install DESTDIR=<staging-root> PREFIX=<final-path>` copies the built `bin/lib/share` content into `$(DESTDIR)$(PREFIX)`. `DESTDIR` is confirmed genuinely native to BSD `make`. `PREFIX`, however, is **not** — testing found zero references to it anywhere in real `bsd.own.mk/bsd.prog.mk/bsd.lib.mk`; real FreeBSD base-system installs use hardcoded individual `*DIR` variables (`BINDIR`, `LIBDIR`, etc.) directly rather than a single unifying `PREFIX` the way GNU autotools or MacPorts do it. Bmake It's `mk.prog.mk/mk.lib.mk` derive those `*DIR` variables from `PREFIX=` themselves²¹. It requires the target to already be built — it will not implicitly trigger a build for you²². **Note:** confirmed by testing against both real FreeBSD and NetBSD's own `share/mk`, BSD `make`'s `install` does not auto-create missing `DESTDIR` parent directories²³ — the destination tree must already exist.

¹⁹REQ-framework-build-order-follows-prereqs-graph-req,REQ-fused-build-per-module-full-dependency-order-req

²⁰REQ-clean-removes-all-derived-artifacts-req

²¹REQ-prefix-is-new-not-reused-req

²²REQ-install-requires-prebuilt-target-req

²³OBS-validation-destdir-native-prefix-is-not

Chapter 4

Advanced Features

4.1 Target Selection and Cross-Compilation

The full target key has the shape `<os>-<arch>[-<toolchain>][-<abi>]`, with each bracketed segment omitted whenever it equals that platform’s default¹:

Segment	Default (omitted)	Shown when non-default
Toolchain	llvm	gcc
ABI/libc	msvc (win); glibc (linux)	mingw, cygwin; musl

On the command line:

```
$ make TARGET=freebsd TARGET_ARCH=amd64 TOOLCHAIN=gcc
```

`TARGET=` and `TARGET_ARCH=` reuse real BSD make’s own FreeBSD cross-build variable names²; `TOOLCHAIN=` is new, since BSD make has no native concept of a toolchain *bundle* (compiler, linker, and archiver selected together)³. Setting `TOOLCHAIN=` includes a dedicated `mk.toolchain.<name>.mk`, which sets `CC/CXX/LD/AR/AS` as a coherent bundle; these remain individually overridable afterward for mixed setups (e.g. Clang with GNU ld)⁴.

A significant, empirically-confirmed correction: unlike most of the variables reused directly from BSD make, `TARGET/TARGET_ARCH` do *not* come with working cross-compilation “for free.” Testing against real FreeBSD `share/mk` showed that setting `TARGET_ARCH=arm64` on an `amd64` host still produced a plain `x86-64` binary — these variables are inert without FreeBSD’s base-system `Makefile.inc1/buildworld` machinery, which a standalone project does not have⁵. Further investigation across the whole BSD-make family (FreeBSD, NetBSD, and generic portable `pmake`) found the identical pattern everywhere: real cross-compilation always happens via an *external orchestration layer*, with `TARGET/TARGET_ARCH`-style variables used only as *data* for architecture-conditional branching, never as a trigger that itself selects a compiler binary. Consequently,

¹REQ-target-key-omit-defaults-req

²REQ-target-arch-toolchain-selection-is-custom-req

³REQ-toolchain-cli-variable-req

⁴REQ-toolchain-triggers-dedicated-mk-file-req

⁵REQ-target-arch-toolchain-selection-is-custom-req

`mk.toolchain.<name>.mk` must itself map (`TARGET`, `TARGET_ARCH`, `TOOLCHAIN`) combinations to concrete compiler paths⁶.

4.2 Resource Files

Both modules and frameworks may carry a `share/` directory for non-source runtime data⁷. Resources support per-target variants via a three-level fallback hierarchy:

Directory	Applies to
<code>share/common/</code>	Every target, unconditionally
<code>share/<os>/</code>	All architectures of one OS (e.g. <code>share/freebsd/</code>)
<code>share/<os>_<arch>/</code>	One specific OS+architecture pairing

Staging is a **layered overlay copy**, not a pick-one-directory selection: `common/` copies first, then `<os>/` on top of it (overwriting same-named files), then `<os>_<arch>/` on top of that⁸. Files unique to any one layer survive regardless; only files present at multiple layers end up as the most specific version. Unlike the compiled-artifact target key, the resource fallback key is **OS+architecture only** — toolchain never affects which resource variant is selected⁹.

4.3 Generated Headers

Headers produced by tooling (yacc/lex output, for example) don’t exist until the build runs, so they can’t live in `include/` as source. They are, by default, **private** to the generating module, landing in `module/build/<KEY>/include/`¹⁰.

A module promotes specific generated headers to public visibility via `INCL=`:

```
INCL=grammar.h
```

On build, the listed headers are copied up into `fw/build/<KEY>/include/` — a framework-level directory containing **only** promoted generated headers, never hand-written ones¹¹. When another framework resolves a `PREREQS`-declared framework’s headers, it checks *both* that framework’s `include/` (hand-written) *and* its `build/<KEY>/include/` (promoted generated) — the one deliberate, scoped exception to “never consult build output for headers”¹². There is no workspace-level equivalent: a workspace-wide generated-header directory is never consulted at compile time, only during packaging.

4.4 Versioned Shared Libraries

Shared-library versioning reuses `SHLIB_MAJOR`/`SHLIB_MINOR` directly, translated per platform by `mk.lib.mk`:

⁶REQ-toolchain-mk-must-set-explicit-cc-paths-req,OBS-validation-cross-compile-is-external-orchestration-everywhere

⁷REQ-resource-file-model

⁸REQ-share-overlay-copy-order

⁹REQ-share-dir-fallback-hierarchy

¹⁰REQ-generated-headers-in-build-tree-req

¹¹REQ-incl-macro-promotes-generated-headers-req

¹²REQ-generated-headers-in-build-tree-req

Platform	Emission
FreeBSD / Linux (ELF)	<code>libfoo.so.MAJOR[.MINOR]</code> + standard symlink chain
macOS (Mach-O)	<code>libfoo.MAJOR.dylib</code> + symlink + linker <code>-compatibility_version/-current_version</code>

A single author-facing macro pair drives both platform-appropriate emission strategies, since embedded Mach-O version metadata (not just the filename) is macOS’s actual ABI-compatibility mechanism¹³. This specific translation for macOS remains **unverified against real MacPorts `bsdmake`** — no macOS environment was available during this project’s validation pass, and real `bsd.lib.mk`’s shared-library logic was designed primarily for ELF; MacPorts’ `bsdmake` may have no Mach-O awareness at all, in which case this logic needs to be written entirely inside `Bmake`. It’s own `mk.lib.mk`. [\[open\]](#)

4.5 Packaging

Packaging is driven by separate named `make` targets, one per format, rather than a single parameterized target¹⁴: confirmed target names so far are `port` (BSD ports-style source recipe) and `pkg` (macOS-style binary package), with `deb/rpm` following the same pattern. Every binary-package target (`pkg`, `deb`, `rpm`) stages a `distrib/<KEY>/` directory — the same target-key scheme as `build/` — by internally invoking the equivalent of `install DESTDIR=distrib/<KEY>`, reusing the install machinery rather than a separate copy mechanism¹⁵.

`make port` is **categorically different**: it produces a source-based, BSD-ports-style recipe (`port Makefile`, `distinfo`, `patches`, `pkg-plist`) for the ports system to fetch and build itself — no pre-built binary is ever staged or embedded¹⁶. Its exact generation mechanism is deferred to a later phase¹⁷. [\[open\]](#)

4.6 Running a Built Program

`make run PROGRAM=<name> ARGS='<args>'`, run at workspace level, sets the platform-appropriate dynamic-library search-path variable (`LD_LIBRARY_PATH` on Linux/FreeBSD, `DYLD_LIBRARY_PATH` on macOS) to include the workspace’s consolidated `build/<KEY>/lib/`, then executes `build/<KEY>/bin/<PROGRAM>` with the given arguments¹⁸ — mirroring real `mkmk`’s own `mkrun` command, adapted to `Bmake`. It’s own `build/<KEY>/` layout.

4.7 Self-Documentation: `make help`

`make help`, run at any level, lists every valid target and variable applicable there, along with a description and — where relevant — the set of valid values (for example, `LIB_SHARED=YES|NO`). Each role-specific `mk.*.mk` file is responsible for documenting its own contribution, so the aggregate output always matches exactly what’s actually `.included` at that level¹⁹.

¹³REQ-shlib-major-minor-cross-platform-emission-req

¹⁴REQ-packaging-per-format-targets-req

¹⁵REQ-packaging-invokes-install-into-distrib-req

¹⁶REQ-port-target-is-source-recipe-not-binary-package-req

¹⁷REQ-make-port-deferred-req

¹⁸REQ-run-target-req

¹⁹REQ-help-target-req

4.8 Convenience Targets

Two small convenience targets exist purely to reduce hand-editing error, without introducing any resolution or search behavior (they do **not** reopen the earlier rejection of `mkGetPreq/mkCopyPreq`):

```
$ make add-prereq FW=System # appends System to PREREQS= in the current framework
$ make add-parent WS=/abs/path # appends /abs/path to PARENT_WS=
```

20

4.9 Dependency-Order Generation

Since real `bsd.subdir.mk` has no concept of a topological build order — it simply processes whatever `SUBDIR` list it's given — `Bmake` It generates that order itself, in a mechanism confirmed by a working, tested prototype rather than only designed on paper²¹:

1. Discover `.m` module directories by scanning.
2. Query each module's `LIBS=` *in isolation*, via `bmake -f <dir>/makefile -V LIBS` — confirmed to read the variable without triggering a build.
3. Topologically sort (Kahn's algorithm), silently ignoring any `LIBS=` entry that isn't a locally-discovered module (such references are resolved via the framework's `PREREQS=` instead, not module-level ordering).
4. Emit the sorted order to a generated `.mk` fragment (`SUBDIR=modA.m modB.m ...`), `.included` before `.include <bsd.subdir.mk>`.

This was tested on a real four-module dependency chain ($A \leftarrow B \leftarrow C \leftarrow D$, with `C` depending on both `A` and `B`) and built in the correct order every time; a genuine cycle ($X \leftrightarrow Y$) was confirmed to fail loudly with a clear diagnostic rather than hanging or silently producing a broken order. The identical mechanism generalizes directly to workspace-level framework ordering, querying `PREREQS=` instead of `LIBS=`.

The order Target

This generation step is formalized as its own explicit, workspace-level target: `make order` performs a **blank run** — no compilation, no linking — scanning every framework and every module across the whole workspace, resolving the complete dependency graph (framework order via `PREREQS=`, and each framework's module order via `LIBS=`) in one pass, and writing the result to a cache²². The default `make` target then reads that cache rather than re-resolving order inline on every invocation.

The order cache is treated as an *implicit prerequisite* of the default build, not an optional manual step: if it is missing or detected stale, `make` regenerates it automatically before proceeding, so a fresh checkout builds correctly with plain `make` and no separate mandatory invocation²³. `make order` remains independently runnable on its own, for inspecting the resolved build order without triggering an actual build.

²⁰REQ-add-prereq-add-parent-convenience-targets-req

²¹REQ-dep-order-generation-mechanism-req

²²REQ-order-target-blank-run-req

²³REQ-order-cache-auto-triggered-req

As a defensive safety net on top of this — since the order cache *should* always be correct, but a stale cache, a partial manual build, or a genuine bug could in principle defeat it — immediately before invoking the linker for a module, **Bmake** It verifies that every **LIBS=** entry’s build artifact actually exists. A missing one produces an explicit, specific error (*“cannot link **hello**: prerequisite library **greet** (from **LIBS=**) has not been built yet”*) rather than the linker’s own generic *“cannot find -lgreet”*²⁴. This should essentially never fire in ordinary operation; its value is a clear diagnostic on the rare occasion something has genuinely gone wrong.

4.10 Caching

Two distinct caches, both modeled on real BSD make’s own **.depend** idiom (a generated file, regenerated via an explicit or detected-stale trigger, consumed via **.include** on subsequent runs)²⁵:

1. **PARENT_WS/PREREQS resolution cache** — workspace/framework-level: caches header search paths and build order so they aren’t re-walked on every invocation. Formalized as an explicit workspace-level target, **make order** — described in Section 4.9.
2. **Per-module .depend** — file-level, **mkdep**-style: which source files include which headers (transitively), so touching a header only triggers recompilation of the affected **.o** files, not a full module rebuild.

The per-module case has a genuinely interesting empirical story, worth telling in full rather than just stating the conclusion. An initial test ran the separate **make depend** target against real FreeBSD **share/mk** and found it nearly useless — no header tracking, just one link-level line. That looked like confirmation the mechanism needed custom shell-script wiring. A closer look reversed that conclusion: every ordinary compile already includes **-MD -MF .depend.<obj>** — **bsd.sys.mk** wires the *compiler’s own* native dependency-generation flag into every build, with no separate step at all, and this was confirmed correct end-to-end (touching a shared header triggered recompilation of exactly the right objects). But a further cross-check against NetBSD’s own bundled **mk** set found *zero* such wiring there — plain compiles, no dependency flags, no rebuild when a header changed. So the working mechanism is real, but specific to FreeBSD’s own actual **bsd.sys.mk**, not a general BSD-make-family property. Since macOS (via MacPorts) is a phase-1 host and MacPorts’ own **bsdmake** could not be tested in this pass, whether it includes equivalent wiring is genuinely unknown²⁶ — though the fallback is simple either way: **mk.prog.mk/mk.lib.mk** can add the same **-MD** flags themselves if MacPorts turns out to lack them.

Exact generated filenames, staleness-detection mechanisms, and regeneration triggers for the **PARENT_WS/PREREQS** cache are deliberately deferred until real **bsd.*.mk** infrastructure can be checked hands-on for what it already provides²⁷. [**open**]

4.11 Empirical Validation Summary

A number of claims in this document rest on empirical testing rather than documentation alone — real **bsd.*.mk** behavior sometimes diverges from reasonable-sounding assumptions, and more than once in this project’s own validation pass, a first test result turned out to need correcting

²⁴REQ-link-time-not-yet-built-guard-req

²⁵REQ-prereqs-resolution-cached-like-depend-req

²⁶REQ-md-dependency-tracking-freebsd-specific-req

²⁷OBS-cache-mechanism-needs-empirical-check

after a closer look. Testing was performed using the portable `bmake` package (a NetBSD-derived, FreeBSD/MacPorts-lineage-compatible `make`) against both a real FreeBSD `share/mk` checkout and NetBSD’s own bundled `mk` set²⁸. A full account, organized by risk tier — including the corrections themselves, not just their final conclusions — is maintained separately as `spec/60-bsd-make-validation-plan.md` the headline results:

Assumption tested	Result
<code>.y/.1</code> suffix rules	Confirmed working automatically, zero explicit rules needed
<code>DESTDIR</code> staging	Confirmed genuinely native, works as expected
<code>DEBUG_FLAGS</code>	Confirmed genuinely native, real working mechanism
Dynamic <code>SUBDIR!=</code> population	Confirmed working — the whole auto-discovery design rests on this
<code>LIB_SHARED=YES NO</code>	Not native — real <code>bsd.lib.mk</code> has no such toggle
<code>TARGET/TARGET_ARCH</code>	Inert without FreeBSD base-system machinery
<code>PREFIX</code>	Not native — does not exist in <code>bsd.prog.mk/bsd.lib.mk</code>
Per-object <code>-MD</code> header dependency tracking	Confirmed genuinely working — but specific to FreeBSD’s own <code>bsd.sys.mk</code> ; absent from NetBSD’s bundled set; MacPorts status unknown

²⁸OBS-validation-yacc-lex-suffix-rules-confirmed

Chapter 5

Tutorial: Building System and Hello From Scratch

This tutorial builds the same worked example used throughout the design documents: two frameworks, **System** (no dependencies) and **Hello** (depends on **System**), where **Hello** contains an executable module `hello.m` and a shared-library module `libgreet.m` that `hello.m` links against¹.

5.1 Step 1 — Create the Workspace

```
$ mkdir -p myworkspace
$ cd myworkspace
$ cat > makefile <<'EOF'
.include <mk.workspace.mk>
PARENT_WS=
EOF
```

An empty `PARENT_WS=` is valid — this workspace has no parents.

5.2 Step 2 — Create the System Framework

```
$ mkdir -p System/include System/local/include
$ cat > System/makefile <<'EOF'
.include <mk.framework.mk>
PREREQS=
EOF
```

The empty, but present, `PREREQS=` is what makes **System** auto-discoverable as a framework at all².

5.3 Step 3 — Create the Hello Framework

```
$ mkdir -p Hello/include Hello/local/include
$ cat > Hello/makefile <<'EOF'
```

¹REQ-framework-contains-modules

²REQ-prereqs-mandatory-even-empty-req

```
.include <mk.framework.mk>
PREREQS=System
EOF
```

5.4 Step 4 — Create the libgreet.m Library Module

```
$ mkdir -p Hello/libgreet.m/src Hello/libgreet.m/include
$ cat > Hello/libgreet.m/makefile <<'EOF'
.include <mk.lib.mk>
LIB=greet
LIB_SHARED=YES
SHLIB_MAJOR=1
LIBS=
EOF
$ cat > Hello/libgreet.m/include/greet.h <<'EOF'
void greet(void);
EOF
$ cat > Hello/libgreet.m/src/greet.c <<'EOF'
#include "greet.h"
#include <stdio.h>
void greet(void) { printf("hello_from_libgreet\n"); }
EOF
```

Note `greet.h` lives in the module's own *private include/* — visible only to `libgreet.m`'s own sources, not even to sibling module `hello.m`, regardless of the fact that `hello.m` will link against it³. If `hello.m` needed to `#include "greet.h"` directly, it would need to live in `Hello's local/include/` instead.

5.5 Step 5 — Create the hello.m Executable Module

```
$ mkdir -p Hello/hello.m/src
$ cat > Hello/hello.m/makefile <<'EOF'
.include <mk.prog.mk>
PROG=hello
LIBS=greet
EOF
$ cat > Hello/hello.m/src/main.c <<'EOF'
extern void greet(void);
int main(void) { greet(); return 0; }
EOF
```

5.6 Step 6 — Build

```
$ cd Hello
$ make
```

Behind the scenes: `libgreet.m` is built first (it has no dependency on `hello.m`), producing `libgreet.so.1` with a `libgreet.so` symlink; `hello.m` is built second, since its `LIBS=greet` entry

³REQ-three-level-header-visibility-req,REQ-libmacro-link-only-headers-via-prereqs-req

makes it depend on `libgreet.m` having been *fully* built first⁴. Both modules' artifacts are then copied up into `Hello/build/<KEY>/{bin,lib}/`⁵.

5.7 Step 7 — Run It

```
$ cd .. # back to the workspace root
$ make run PROGRAM=hello
hello from libgreet
```

`make run` sets `LD_LIBRARY_PATH` (or `DYLD_LIBRARY_PATH` on macOS) to include the workspace's consolidated `build/<KEY>/lib/` before executing, so `libgreet.so` is found without needing a real `make install` step first⁶.

5.8 Step 8 — Clean Up

```
$ make clean TARGET=all
```

This removes every generated artifact for every target present — the whole `build/` tree, any `distrib/` packaging staging, and all internal caches — leaving the workspace indistinguishable from a fresh checkout⁷.

This document cites the discovery-driven design record maintained in `project-model/`. Every footnoted alias (e.g. `REQ-framework-contains-modules`) corresponds to exactly one file there, carrying its full rationale, decision history, and any superseded predecessors.

⁴REQ-fused-build-per-module-full-dependency-order-req

⁵REQ-aggregation-step-copies-outputs

⁶REQ-run-target-req

⁷REQ-clean-removes-all-derived-artifacts-req